HEART DISEASE RISK PREDICTION

MLOps Final Report

Course	AIMLCZG523 — MLOps
Assignment	01
Dataset	UCI Heart Disease (Cleveland)
Name	BADRINATHAN S.
Student ID	2025AE05369
Email	2025ae05369@wilp.bits-pilani.ac.in
Git Repo	https://github.com/2025ae05369-badree/heart-disease-prediction-mlops.git
Google drive link	https://wilpbitspilaniacin0-my.sharepoint.com/:f:/g/personal/2025ae05369_wilp_bits-pilani_ac_in/IgBM36qUB9bsS4vpV3o4zAECAew-QsCe5ayX4JSdlY00rq4?e=PvuaY0 (Folder containing project files and video recordings)

This report documents the complete lifecycle of a binary heart-disease risk classifier: data preparation, exploratory analysis, model development, experiment tracking, API serving, containerization, CI, Kubernetes deployment, monitoring, and evidence-based verification.

1. Executive Summary and Architecture

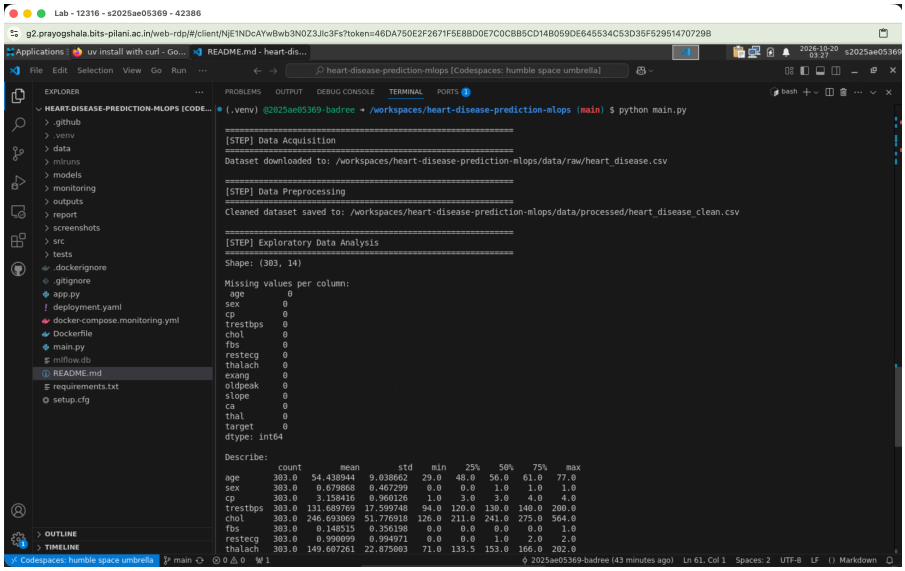
The project delivers a reproducible machine-learning service that predicts whether a patient is likely to have heart disease. It uses 13 clinical input features, a preprocessing-and-model pipeline, and a FastAPI interface. Logistic Regression was selected after comparison with Random Forest because it achieved the strongest holdout performance while maintaining the same recall.

System architecture

Stage	Implementation	Operational outcome
Data	UCI Cleveland CSV	303 cleaned patient records
Preparation	Median/mode imputation; binary target	Consistent training matrix
Model	Logistic Regression and Random Forest	Tracked comparison and selection
Serving	FastAPI with typed request/response models	Health, prediction, and metrics endpoints
Packaging	Docker image; Kubernetes Deployment and Service	Portable local deployment
Observability	Prometheus and Grafana	Request, latency, success, and error visibility

Reproducibility

The implementation is managed with a Python virtual environment. Running python main.py downloads the data, performs preprocessing and EDA, trains both classifiers, records MLflow experiments, and persists the selected model. Dependencies are pinned in requirements.txt and setup and execution instructions are provided in the repository README.



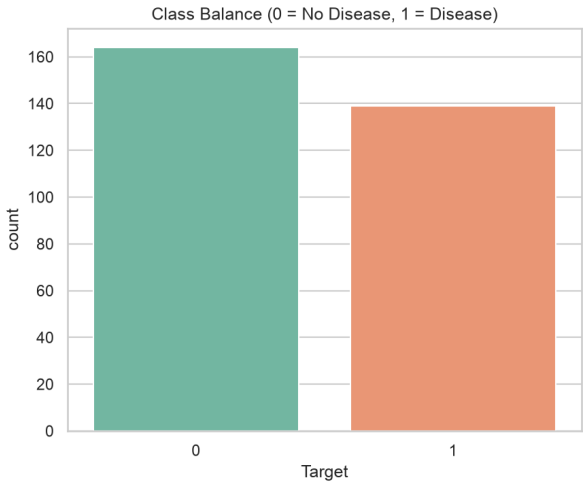
2. Data Preparation and Exploratory Analysis

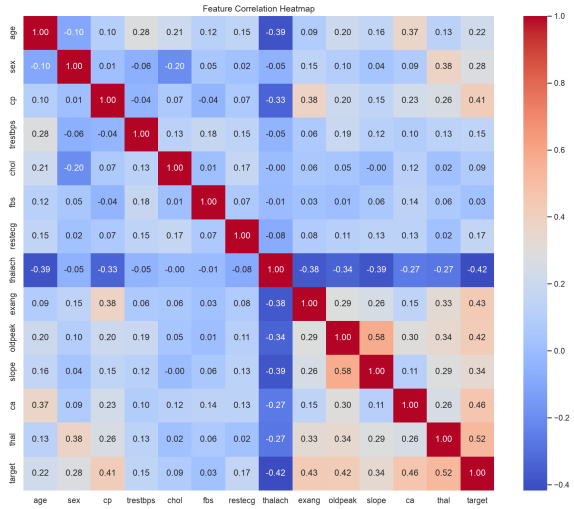
The source is the UCI Machine Learning Repository Cleveland heart-disease dataset (processed.cleveland.data). The dataset contains 303 rows and 13 clinical features. Question-mark values are treated as missing values. Numeric fields are imputed with medians and categorical fields with modes, producing a complete modeling dataset.

Data characteristic	Result
Observations	303
Predictor features	13
Negative class	164
Positive class	139
Missing values after cleaning	0
Target transformation	Severity 0–4 mapped to 0/1

Exploratory findings

- The target is moderately balanced, with 164 negative and 139 positive records.
- Continuous variables include age, resting blood pressure, cholesterol, maximum heart rate, and exercise-induced ST depression.
- Categorical variables capture sex, chest-pain type, fasting blood sugar, resting ECG, exercise angina, slope, number of vessels, and thalassemia.
- The EDA outputs include distributions, a correlation heatmap, and class-balance visualizations.





3. Feature Engineering and Model Development

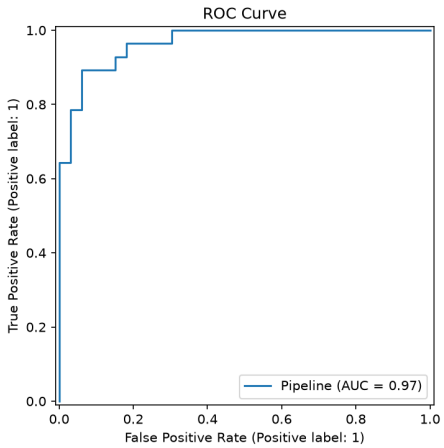
A scikit-learn ColumnTransformer keeps preprocessing inside the persisted pipeline. Standard scaling is applied to age, trestbps, chol, thalach, and oldpeak. One-hot encoding is applied to the categorical features. This prevents training-serving skew because the same transformations are applied during API inference.

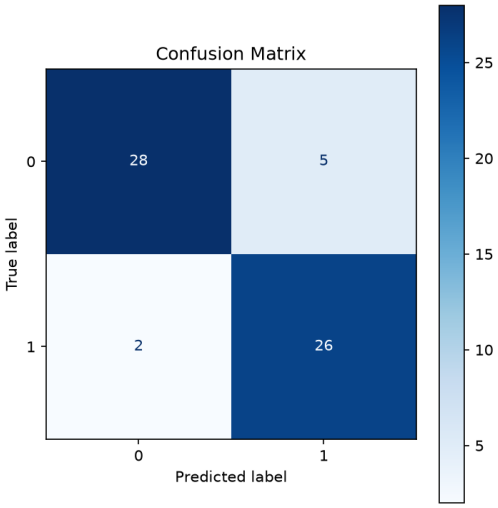
Model selection design

Logistic Regression and Random Forest were tuned with five-fold stratified cross-validation. ROC-AUC was used as the optimization metric, and final metrics were measured on a stratified 80/20 holdout set. The comparison emphasizes discrimination, accuracy, precision, and recall rather than relying on a single score.

Model	CV ROC-AUC	Test ROC-AUC	Accuracy	Precision	Recall
Logistic Regression	0.9025 ± 0.0144	0.9665	0.8852	0.8387	0.9286
Random Forest	0.9024 ± 0.0279	0.9470	0.8689	0.8125	0.9286

Logistic Regression was selected because it produced the better holdout ROC-AUC, accuracy, and precision, matched Random Forest on recall, and showed lower cross-validation variance. In a health-risk screening context, retaining recall while improving precision is operationally useful because it limits missed positive cases without unnecessarily increasing false alarms.



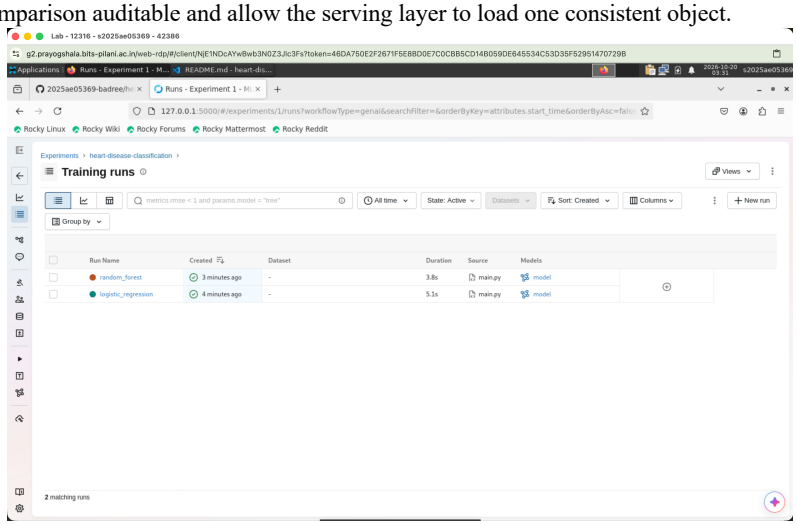


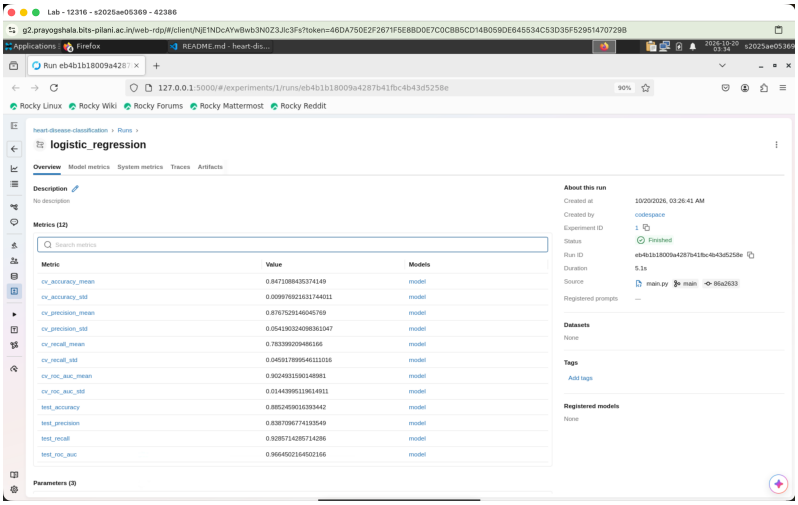
4. Experiment Tracking and Model Packaging

MLflow runs locally under the heart-disease-classification experiment. Each run records tuned parameters, cross-validation and holdout metrics, the fitted preprocessing-and-model pipeline, and visual artifacts such as confusion matrices and ROC curves.

Packaging decisions

- The selected pipeline is saved as models/heart_disease_model.joblib.
- Preprocessing and prediction are persisted as one artifact, preventing transformation drift between training and serving.
- The dependency environment is captured through pinned requirements and MLflow model metadata.
- Artifacts make model comparison auditable and allow the serving layer to load one consistent object.





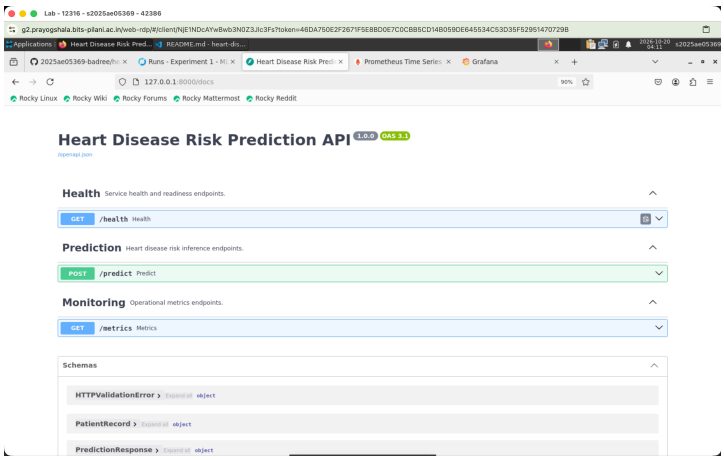
5. API Service and Containerization

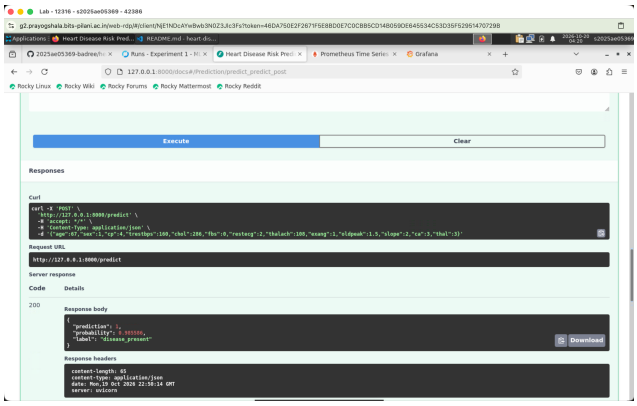
The FastAPI service exposes a small, typed contract for operational use. PatientRecord validates all 13 input features, while the response model preserves the integer prediction, probability, and human-readable label.

Endpoint	Purpose	Expected behavior
GET /health	Model readiness	Confirms the model is available
POST /predict	Inference	Returns prediction, probability, and label
GET /metrics	Observability	Exposes Prometheus metrics

Container boundary

The Docker image uses Python 3.12, installs pinned dependencies, and serves the API on port 8000. Keeping the model artifact and preprocessing pipeline together makes the image self-contained and repeatable across local and Kubernetes environments.



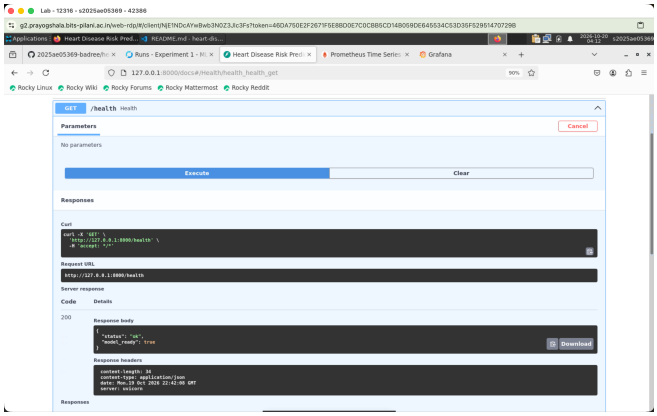


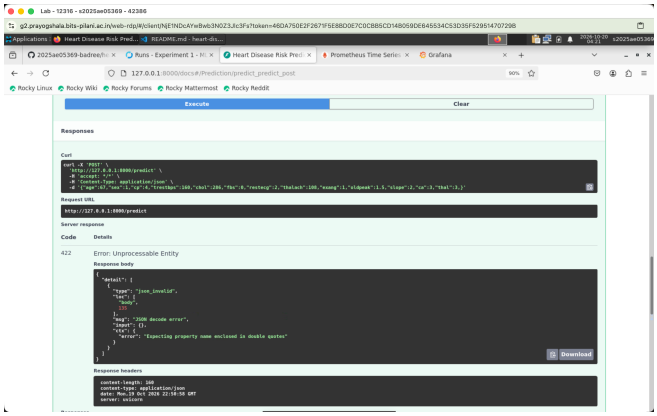
6. API Validation and Automated Testing

The API evidence covers readiness, valid predictions, invalid request handling, and metrics exposure. FastAPI's validation layer returns an Unprocessable Entity response for malformed input, making contract failures explicit to clients.

Observed service behaviors

- The health endpoint confirms that the persisted model can be loaded.
- The prediction endpoint returns a result and probability for valid patient records.
- The service distinguishes no-disease and disease-present cases through the response label.
- Invalid or incomplete request payloads are rejected with a structured validation error.
- Prometheus instrumentation exposes request counts and latency histograms.





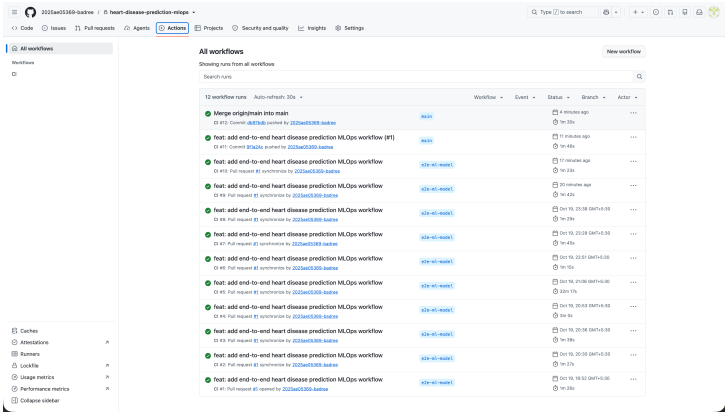
Continuous integration

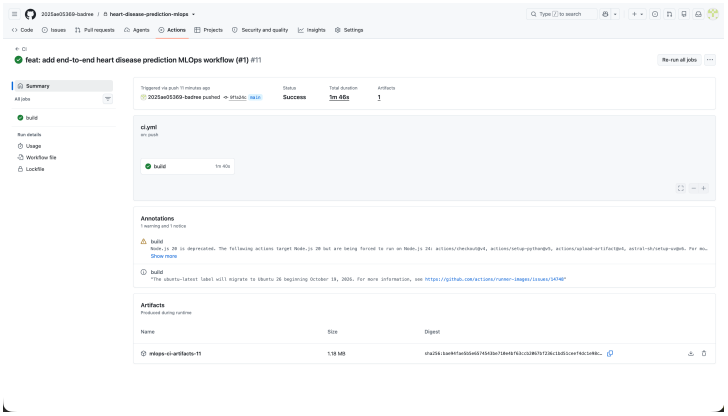
The GitHub Actions workflow runs flake8, pytest -q, and python main.py on pushes and pull requests to the main branches. This sequence checks code quality, tests behavior, and verifies that the end-to-end pipeline can execute.

7. CI Workflow and Deployment

CI verification

Automated checks provide a repeatable quality gate before integration. Linting catches maintainability issues, pytest verifies data and service behavior, and the complete pipeline execution validates that the project remains runnable from source.



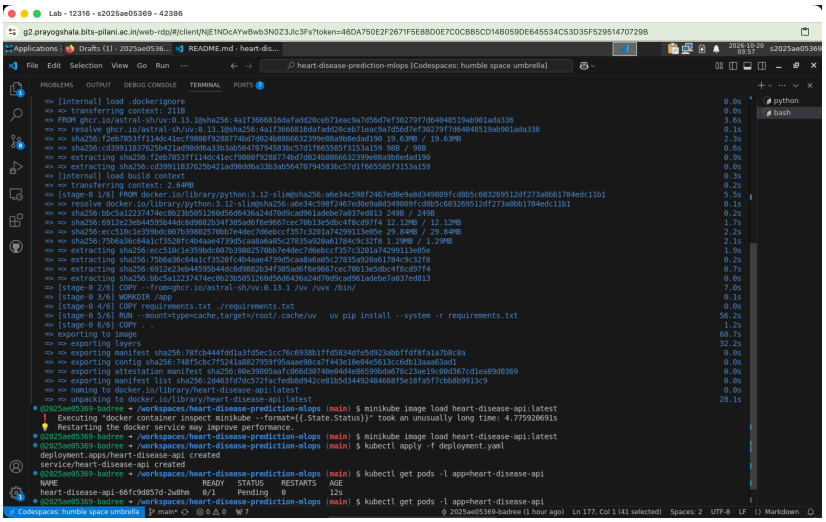


Kubernetes deployment

deployment.yaml defines one API replica and a Service that maps port 80 to container port 8000. The Docker image can be loaded into Minikube, and kubectl port-forward provides local access for verification.

Deployment check	Verification target
Node readiness	A ready Kubernetes node
Pod readiness	A running API pod

Service reachability	A reachable Kubernetes Service
Functional smoke test	Successful health and prediction requests

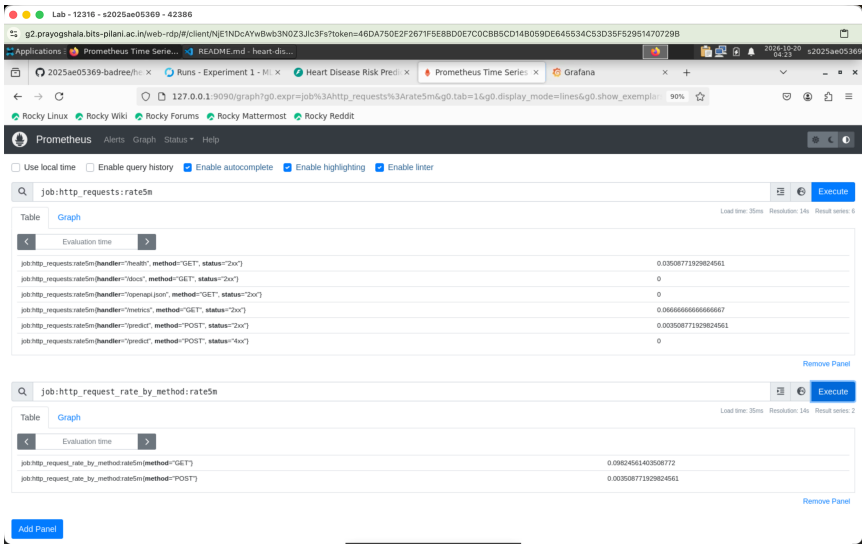


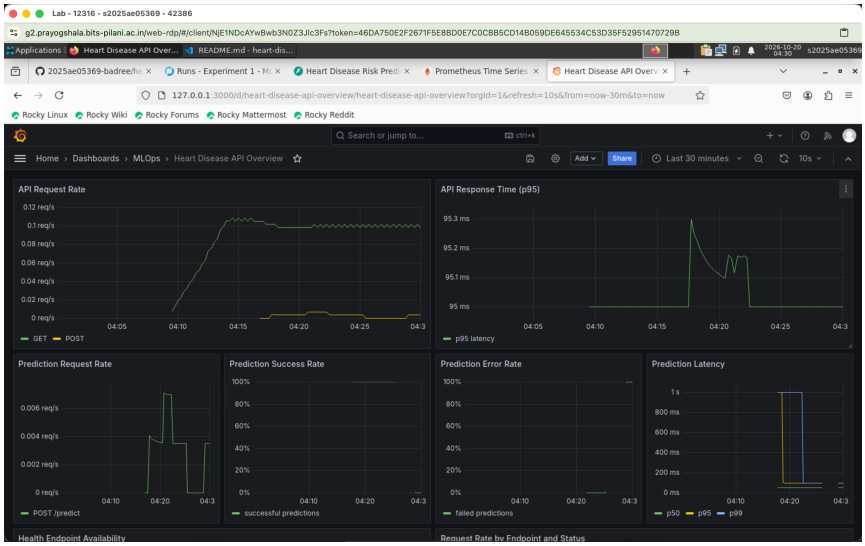
8. Monitoring and Logging

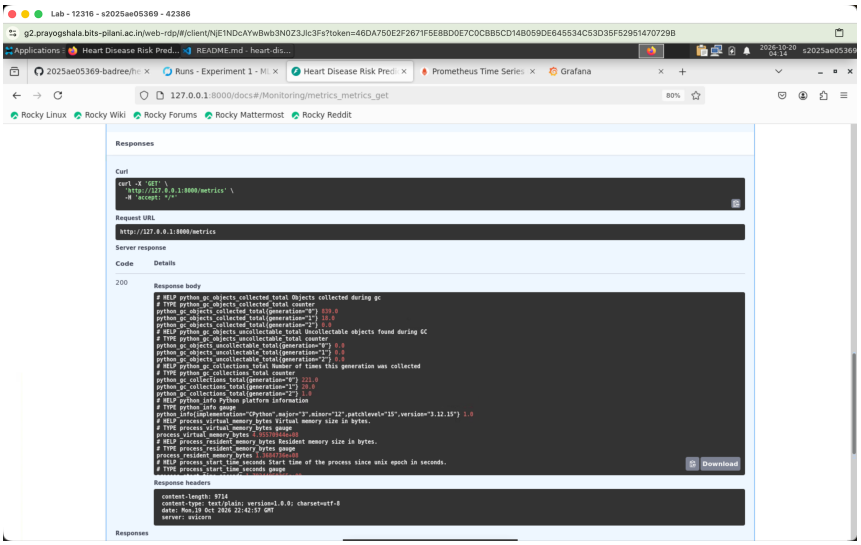
The API uses Python logging for health and prediction requests, while Uvicorn provides access logs. prometheus-fastapi-instrumentator exposes request counts and latency histograms at /metrics. The Compose monitoring stack adds Prometheus collection and a provisioned Grafana dashboard.

Operational signals

- Request rate and total request counts show service demand.
- Latency histograms reveal response-time behavior.
- Success and error rates provide a basic service-health view.
- Prediction-specific queries separate model traffic from other API traffic.
- Grafana dashboards provide a consolidated operational view for local deployment.







9. Evidence Register and Reproducibility Checklist

Evidence category	Location
Environment and execution	screenshots/version-check/; screenshots/e2e-model-execution/
EDA outputs	outputs/figures/
Model evaluation	outputs/figures/mlflow_runs/
MLflow comparison	screenshots/mlflow-run/
API health and prediction	screenshots/fastapi-service/
CI workflow	screenshots/ci-run/
Kubernetes deployment	screenshots/kubernetes/
Monitoring	screenshots/prometheus/; screenshots/grafana/

Reproducibility checklist

- Install the pinned dependencies in a Python virtual environment.
- Run python main.py to download data, clean it, generate EDA, train models, and persist the selected pipeline.
- Review MLflow runs and artifacts for parameters, metrics, and plots.
- Build and run the Docker image, then exercise /health, /predict, and /metrics.
- Deploy the image and deployment.yaml to a ready local Kubernetes cluster.
- Start Prometheus and Grafana, then confirm request and latency signals.

10. Conclusion

This project demonstrates an end-to-end MLOps implementation for a heart-disease risk classifier. The solution begins with reproducible data preparation and EDA, compares two candidate models with cross-validation and holdout evaluation, and selects Logistic Regression using transparent performance criteria.

The selected model is packaged with its preprocessing logic, tracked through MLflow, exposed through a validated FastAPI contract, and containerized for deployment. GitHub Actions provides automated lint, test, and pipeline checks. Kubernetes supplies a repeatable deployment target, while Prometheus and Grafana make API behavior observable.

Final outcome

Capability	Delivered result
Predictive model	Logistic Regression; test ROC-AUC 0.9665
Data quality	303 records; no missing values after cleaning
API	Health, prediction, and Prometheus metrics endpoints
Automation	Linting, tests, and end-to-end execution in CI
Deployment	Docker image and Kubernetes Deployment/Service
Observability	Prometheus queries, Grafana dashboard, and logs